

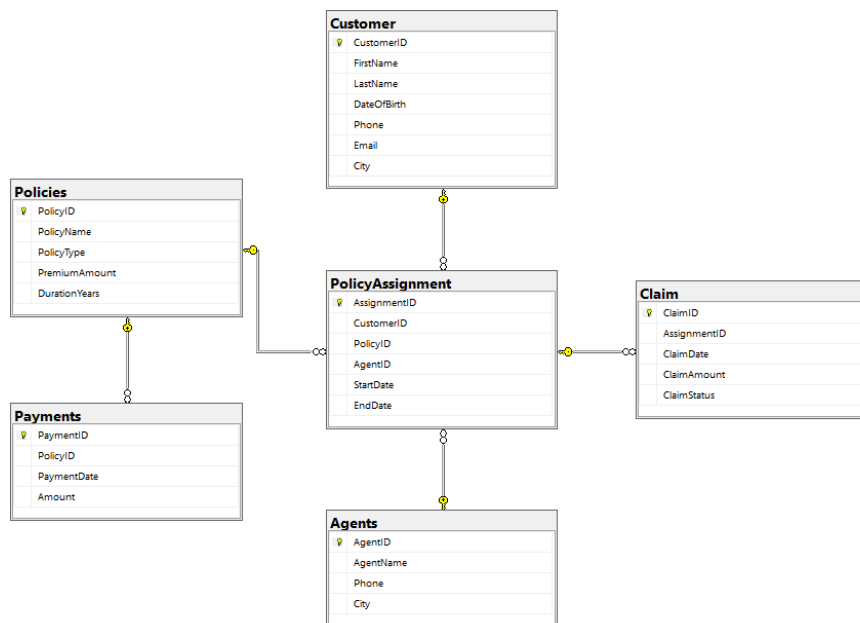
MODULE-4.4 Practical Project Assignment

29-12-2025 (Day-2)

InsuranceDB:

1) Database creation

```
CREATE DATABASE Insurance;  
USE Insurance;
```



2) DDL

```
CREATE TABLE Customer(  
    CustomerID INT PRIMARY KEY,  
    FirstName VARCHAR(50),  
    LastName VARCHAR(50),  
    DateOfBirth DATE,  
    Phone INT,  
    Email VARCHAR(50)  
)  
  
CREATE TABLE Policies(  
    PolicyID VARCHAR(25) PRIMARY KEY,  
    PolicyName VARCHAR(50),  
    PolicyType VARCHAR(50),  
    PremiumAmount MONEY,  
    DurationYears VARCHAR(20)  
)
```

```

CREATE TABLE Agents(
    AgentID VARCHAR(50) PRIMARY KEY,
    AgentName VARCHAR(50),
    Phone INT,
    City VARCHAR(50)
)
CREATE TABLE PolicyAssignment(
    AssignmentID VARCHAR(50) PRIMARY KEY,
    CustomerID INT,
    PolicyID VARCHAR(25),
    AgentID VARCHAR(50),
    StartDate DATE,
    EndDate DATE,
    CONSTRAINT fk_customer_assign
    FOREIGN KEY (CustomerID) REFERENCES Customer(CustomerID),
    CONSTRAINT fk_policy_assign
    FOREIGN KEY (PolicyID) REFERENCES Policies(PolicyID),
    CONSTRAINT fk_agent_assign
    FOREIGN KEY (AgentID) REFERENCES Agents(AgentID)
)
CREATE TABLE Claim(
    ClaimID VARCHAR(50) PRIMARY KEY,
    AssignmentID VARCHAR(50),
    ClaimDate DATE,
    ClaimAmount MONEY,
    ClaimStatus VARCHAR(20),
    CONSTRAINT fk_assign_claims
    FOREIGN KEY (AssignmentID) REFERENCES PolicyAssignment(AssignmentID),
)
CREATE TABLE Payments(
    PaymentID VARCHAR(25) PRIMARY KEY,
    PolicyID VARCHAR(25),
    PaymentDate DATE,
    Amount MONEY,
    ClaimStatus VARCHAR(20),
    CONSTRAINT fk_payments_policies
    FOREIGN KEY (PolicyID) REFERENCES Payments(PolicyID),
)

```

3) Data Insertion

```

INSERT INTO Customer (CustomerID, FirstName, LastName, DateOfBirth, Phone, Email)
VALUES
(101, 'Rahul', 'Sharma', '1990-05-10', 987654321, 'rahul.sharma@email.com'),
(102, 'Priya', 'Mehta', '1988-11-22', 912345678, 'priya.mehta@email.com'),
(103, 'Vikram', 'Rao', '1995-03-15', 998877665, 'vikram.rao@email.com'),
(104, 'Ananya', 'Iyer', '1992-08-09', 909090909, 'ananya.iyer@email.com');

```

```

INSERT INTO Policies (PolicyID, PolicyName, PolicyType, PremiumAmount,
DurationYears)
VALUES
('P001', 'Health Secure', 'Health', 12000, '10'),
('P002', 'Life Protect', 'Life', 25000, '20'),
('P003', 'Car Shield', 'Vehicle', 8000, '5'),
('P004', 'Home Guard', 'Property', 15000, '15');

```

```

INSERT INTO Agents (AgentID, AgentName, Phone, City)
VALUES
('A101', 'Rohit Verma', 987651234, 'Mumbai'),
('A102', 'Sneha Kapoor', 998866554, 'Delhi'),
('A103', 'Amit Singh', 909988776, 'Hyderabad');

INSERT INTO PolicyAssignment
(AssignmentID, CustomerID, PolicyID, AgentID, StartDate, EndDate)
VALUES
('AS001', 101, 'P001', 'A101', '2022-01-01', '2032-01-01'),
('AS002', 101, 'P003', 'A103', '2023-05-10', '2028-05-10'),
('AS003', 102, 'P002', 'A102', '2021-03-15', '2041-03-15'),
('AS004', 103, 'P004', 'A101', '2024-02-01', '2039-02-01');

INSERT INTO Claim (ClaimID, AssignmentID, ClaimDate, ClaimAmount, ClaimStatus)
VALUES
('C001', 'AS001', '2023-02-15', 50000, 'Approved'),
('C002', 'AS002', '2024-08-10', 20000, 'Pending'),
('C003', 'AS003', '2022-12-01', 75000, 'Rejected'),
('C004', 'AS001', '2024-01-20', 30000, 'Approved');

```

QUERIES

A. Comparison Operators (=, <>, >, <, >=, <=)

```

/*1. Write a query to find all Agents who live in 'Delhi'*/
SELECT * FROM Agents WHERE City='Delhi';

/*2. Display policies with PremiumAmount greater than 10,000.*/
SELECT * FROM Policies WHERE PremiumAmount>10000;

/*3. List claims where ClaimAmount is less than 50,000.*/
SELECT * FROM Claim WHERE ClaimAmount<50000;

/*4. Find Agents whose city is not 'Delhi'.*/
SELECT * FROM Agents WHERE City NOT IN ('Delhi');

/*5. Display policies that expire on or before '2040-12-31'.*/
SELECT p.PolicyName,p.PolicyType,pa.EndDate FROM Policies p
JOIN PolicyAssignment pa
ON pa.PolicyID=p.PolicyID
WHERE pa.EndDate<'2040-12-31';

/*6. Retrieve Claims after 1st Jan 2023. */
SELECT * FROM Claim WHERE ClaimDate>'2023-01-01';

```

B. Logical Operators (AND, OR, NOT)

```

/*7. Find policies where PolicyType is 'Health' AND PremiumAmount > 10,000.*/
SELECT * FROM Policies WHERE PolicyType='Health' AND PremiumAmount>10000;

/*8. List Agents who live in Mumbai OR Hyderabad.*/
SELECT * FROM Agents WHERE City='Hyderabad' OR City='Mumbai';

/*9. Display claims where ClaimStatus is NOT 'Approved'.*/
SELECT * FROM Claim WHERE ClaimStatus!='Approved';

```

/*10. Find policies that have more than 10 years duration AND PremiumAmount < 26,000.*/

```
SELECT * FROM Policies WHERE DurationYears>10 AND PremiumAmount<26000;
```

/*11. Find Customers Older Than 30 Years and Name ends with a*/

```
SELECT * FROM Customer WHERE DATEDIFF(YEAR, DateOfBirth, GETDATE()) > 30 AND  
FirstName LIKE '%a' ;
```

/*12. Show agents whose name has atleast 10 characters OR live in 'Delhi'.*/

```
SELECT * FROM Agents WHERE LEN(AgentName) >= 10 OR City='Delhi';
```

C. IN Operator

/*13. Display agents whose City is IN ('Delhi', 'Mumbai', 'Hyderabad').*/

```
SELECT * FROM Agents WHERE City IN ('Delhi', 'Mumbai', 'Hyderabad');
```

/*14. Find policies with PolicyType IN ('Life', 'Health').*/

```
SELECT * FROM Policies WHERE PolicyType IN ('Health', 'Life');
```

/*15. Retrieve claims where ClaimStatus IN ('Rejected', 'Pending').*/

```
SELECT * FROM Claim WHERE ClaimStatus IN ('Rejected', 'Pending');
```

/*16. Show policy assignments handled by agents whose IDs are IN ('A101', 'A103').*/

```
SELECT * FROM PolicyAssignment WHERE AgentID IN ('A101', 'A103');
```

D. BETWEEN Operator

/*17. Find policies with PremiumAmount BETWEEN 10,000 AND 30,000.*/

```
SELECT * FROM Policies WHERE PremiumAmount BETWEEN 10000 AND 30000;
```

/*18. Retrieve claims made between '2024-01-01' and '2024-12-31'.*/

```
SELECT * FROM Claim WHERE ClaimDate BETWEEN '2024-01-01' AND '2024-12-31';
```

/*19. List customers born between '1985-01-01' and '1995-12-31'.*/

```
SELECT * FROM Customer WHERE DateOfBirth BETWEEN '1985-01-01' AND '1995-12-31'
```

/*20. Display payments where PaymentAmount BETWEEN 5,000 AND 20,000.*/

```
SELECT * FROM Payments WHERE Amount BETWEEN 5000 AND 20000;
```

E. LIKE Operator (Pattern Matching)

/*21. Find customers whose FirstName starts with 'A'.*/

```
SELECT * FROM Customer WHERE FirstName LIKE 'A%';
```

/*22. Display agents whose AgentName ends with 'Sharma'.*/

```
SELECT * FROM Agents WHERE AgentName LIKE '%Sharma';
```

/*23. Retrieve customers whose LastName contains 'Iyer'.*/

```
SELECT * FROM Customer WHERE LastName LIKE '%Iyer%';
```

/*24. List cities from Agents where City starts with 'M'.*/

```
SELECT * FROM Agents WHERE City LIKE 'M%';
```

/*25. Find policies where PolicyType contains the word 'Vehicle'. */

```
SELECT * FROM Policies WHERE PolicyType LIKE '%Vehicle%';
```

F. IS NULL / IS NOT NULL

```
/*26. Display policies where EndDate IS NULL.*/  
SELECT p.PolicyName,p.PolicyType,pa.EndDate FROM Policies p  
JOIN PolicyAssignment pa  
ON pa.PolicyID=p.PolicyID  
WHERE pa.EndDate IS NULL;
```

```
/*27. Find claims where ClaimStatus IS NOT NULL.*/  
SELECT * FROM Claim WHERE ClaimStatus IS NOT NULL;
```

```
/*28. Retrieve Agent where City IS NOT NULL. */  
SELECT * FROM Agents WHERE City IS NOT NULL;
```

G. Mixed / Tricky Scenarios

```
/*29. Find policies where PolicyType = 'Health' AND PremiumAmount BETWEEN 10,000  
AND 15,000.*/  
SELECT * FROM Policies WHERE PolicyType='Health' AND PremiumAmount BETWEEN 10000  
AND 15000;
```

```
/*30. Display claims where ClaimAmount < 1,00,000 AND ClaimStatus <>  
'Rejected'.*/  
SELECT * FROM Claim WHERE ClaimAmount<100000 AND ClaimStatus='Rejected';
```

Functions in Querying

STRING FUNCTION QUESTIONS (1-10)

```
/*1. Display customer full name by concatenating FirstName and LastName.*/  
SELECT CONCAT(FirstName, ' ', LastName) AS FullName FROM Customer;
```

```
/*2. Find customers whose FirstName starts with 'A'.*/  
SELECT * FROM Customer WHERE FirstName LIKE 'A%';
```

```
/*3. Display LastName in UPPERCASE.*/  
SELECT UPPER(LastName) AS LastName FROM Customer;
```

```
/*4. Find customers whose LastName length is more than 4 characters.*/  
SELECT * FROM Customer WHERE LEN(LastName)>4;
```

```
/*5. Display the first 3 characters of FirstName.*/  
SELECT LEFT(FirstName, 3) AS FirstName FROM Customer;
```

```
/*6. Replace the word 'Life' with 'Term Life' in PolicyType.*/  
SELECT REPLACE(PolicyType, 'Life', 'Term Life') AS PolicyType FROM Policies;
```

```
/*7. Find customers whose City contains 'del' (case-insensitive).*/  
SELECT * FROM Agents WHERE LOWER(City) LIKE '%del%'
```

```
/*8. Remove leading and trailing spaces from customer FirstName.*/  
SELECT LTRIM(FirstName) AS FirstName FROM Customer;
```

```
/*9. Display PolicyType and its character length.*/  
SELECT PolicyType, LEN(PolicyType) AS Length FROM Policies;
```

```
/*10. Extract last 4 characters from PolicyType. */  
SELECT PolicyType, RIGHT(PolicyType, 4) AS PolicyTypeL4 FROM Policies;
```

NUMERIC FUNCTION QUESTIONS (11–20)

```
/*11. Display PremiumAmount rounded to nearest integer.*/  
SELECT ROUND(PremiumAmount, 0) AS PremiumAmount FROM Policies  
  
/*12. Find policies where PremiumAmount is greater than 5000 after rounding.*/  
SELECT * FROM Policies WHERE ROUND(PremiumAmount, 0) > 5000;  
  
/*13. Display absolute value of ClaimAmount.*/  
SELECT ABS(ClaimAmount) AS ClaimAmount FROM Claim;  
  
/*14. Calculate 10% tax on PremiumAmount.*/  
SELECT 0.1 * PremiumAmount AS Tax FROM Policies;  
  
/*15. Display PremiumAmount rounded to 2 decimal places.*/  
SELECT ROUND(PremiumAmount, 2) AS PremiumAmount FROM Policies;  
  
/*16. Find the square root of PremiumAmount.*/  
SELECT SQRT(PremiumAmount) AS SQRTOfPremium FROM Policies  
  
/*17. Display the ceiling value of ClaimAmount.*/  
SELECT CEILING(ClaimAmount) AS Amount FROM Claim;  
  
/*18. Display the floor value of PremiumAmount.*/  
SELECT FLOOR(PremiumAmount) AS Amount FROM Policies;  
  
/*19. Calculate total premium including 18% GST.*/  
SELECT 1.18 * PremiumAmount AS INCGSTPremium FROM Policies;  
  
/*20. Find policies where ClaimAmount modulo 2 = 0.*/  
SELECT * FROM Claim WHERE ClaimAmount % 2 = 0;
```

DATE FUNCTION QUESTIONS (21–30)

```
/*21. Display current system date.*/  
SELECT GETDATE();  
  
/*22. Find policies that started in the year 2023.*/  
SELECT * FROM PolicyAssignment WHERE YEAR(StartDate) = 2023;  
  
/*23. Calculate policy duration in days.*/  
SELECT AssignmentID, StartDate, EndDate, DATEDIFF(DAY, StartDate, EndDate) AS  
PolicyDurationDays FROM PolicyAssignment;  
  
/*24. Find customers whose birthday is in the current month.*/  
SELECT * FROM Customer WHERE MONTH(DateOfBirth) = MONTH(GETDATE());  
  
/*25. Display ClaimDate in DD-MM-YYYY format.*/  
SELECT ClaimID, CONVERT(VARCHAR(10), ClaimDate, 105) AS FormattedClaimDate FROM  
Claim;  
  
/*26. Find policies that expire within the next 30 days.*/  
SELECT * FROM PolicyAssignment WHERE EndDate BETWEEN GETDATE() AND DATEADD(DAY,  
30, GETDATE());  
  
/*27. Calculate customer age from DOB.*/  
SELECT DATEDIFF(YEAR, DateOfBirth, GETDATE()) AS Age FROM Customer;  
  
/*28. Find claims filed in the last 6 months.*/  
SELECT * FROM Claim WHERE ClaimDate >= DATEADD(MONTH, -6, GETDATE());
```

```
/*29. Display month name from Policy StartDate.*/  
SELECT AssignmentID,StartDate,DATENAME(MONTH, StartDate) AS MonthName FROM  
PolicyAssignment;
```

```
/*30. Find policies where StartDate is a Monday. */  
SELECT * FROM PolicyAssignment WHERE DATENAME(WEEKDAY, StartDate) = 'Monday';
```

AGGREGATE FUNCTION QUESTIONS (31–40)

```
/*31. Find the total number of customers.*/  
SELECT COUNT(CustomerID) AS TotalNumberOfCustomers FROM Customer;
```

```
/*32. Calculate total premium collected.*/  
SELECT SUM(PremiumAmount) AS TotalPremiumCollected FROM Policies;
```

```
/*33. Find average premium amount.*/  
SELECT AVG(PremiumAmount) AS AVGPremiumCollected FROM Policies;
```

```
/*34. Find the maximum claim amount.*/  
SELECT MAX(PremiumAmount) AS MAXPremiumCollected FROM Policies;
```

```
/*35. Find the minimum premium amount.*/  
SELECT MIN(PremiumAmount) AS MINPremiumCollected FROM Policies;
```

```
/*36. Count number of policies per PolicyType.*/  
SELECT PolicyType,COUNT(*) AS NumberOfPolicies FROM Policies GROUP BY PolicyType;
```

```
/*37. Find total ClaimAmount per ClaimStatus.*/  
SELECT ClaimStatus,SUM(ClaimAmount) AS ClaimAmount FROM Claim GROUP BY  
ClaimStatus;
```

```
/*38. Find PolicyTypes where average premium > 6000.*/  
SELECT PolicyType,AVG(PremiumAmount) AS AveragePremium FROM Policies GROUP BY  
PolicyType HAVING AVG(PremiumAmount)>6000;
```

```
/*39. Count number of claims per year.*/  
SELECT YEAR(ClaimDate) AS ClaimYear,COUNT(*) AS NumberOfClaims  
FROM Claim  
GROUP BY YEAR(ClaimDate)  
ORDER BY ClaimYear;
```

```
/*40. Find customers having more than 1 policies.*/  
SELECT CustomerID,COUNT(*) AS NumberOfPolicies  
FROM PolicyAssignment  
GROUP BY CustomerID  
HAVING COUNT(*) >1;
```

Joins in Querying

/*V Queries using Joins, Group By, Having etc.*/

/*1. List all Policies for a CustomerId 5.*/

```
SELECT p.PolicyID, p.PolicyName, p.PolicyType, p.PremiumAmount, p.DurationYears
FROM PolicyAssignments pa
JOIN Policies p
ON pa.PolicyID=p.PolicyID
WHERE pa.CustomerID=5;
```

/*2. View all customers with their policies.*/

```
SELECT c.CustomerID, c.FirstName, c.LastName, p.PolicyID, p.PolicyName,
p.PolicyType, p.PremiumAmount
FROM Customers c
JOIN PolicyAssignments pa
ON c.CustomerID = pa.CustomerID
JOIN Policies p
ON pa.PolicyID = p.PolicyID;
```

/*3. View claims with customer name.*/

```
SELECT c.ClaimID, c.ClaimDate, c.ClaimAmount, c.ClaimStatus, CONCAT(cu.FirstName, '
', cu.LastName) AS CustomerName
FROM Claims c
JOIN PolicyAssignments pa
ON c.AssignmentID = pa.AssignmentID
JOIN Customers cu
ON pa.CustomerID = cu.CustomerID;
```

/*4. Display FirstName, PolicyName, AgentName, StartDate and EndDate from their respective tables.*/

```
SELECT c.FirstName, p.PolicyName, a.AgentName, pa.StartDate, pa.EndDate
FROM Customers c
JOIN PolicyAssignments pa
ON c.CustomerID=pa.CustomerID
JOIN Agents a
ON a.AgentID=pa.AgentID
JOIN Policies p
ON p.PolicyID=pa.PolicyID;
```

/*5. Display claims report with FirstName, PolicyName, ClaimAmount, ClaimStatus, and ClaimDate from their respective tables.*/

```
SELECT c.FirstName, p.PolicyName, cl.ClaimAmount, cl.ClaimStatus, cl.ClaimDate
FROM Customers c
JOIN PolicyAssignments pa
ON c.CustomerID=pa.CustomerID
JOIN Claims cl
ON cl.AssignmentID=pa.AssignmentID
JOIN Policies p
ON p.PolicyID=pa.PolicyID;
```

/*6. Display records of Customers with or without Policies.*/

```
SELECT c.CustomerID, CONCAT(c.FirstName, ' ', c.LastName) AS
CustomerName, pa.PolicyID
FROM Customers c
LEFT JOIN PolicyAssignments pa
ON c.CustomerID=pa.CustomerID;
```



```

/*7. Display all Customers with NO Claims.*/
SELECT DISTINCT c.CustomerID, CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName
FROM Customers c
LEFT JOIN PolicyAssignments pa
ON c.CustomerID=pa.CustomerID
LEFT JOIN Claims cl
ON pa.AssignmentID=cl.AssignmentID
WHERE cl.ClaimID IS NULL;

```

```

/*8. Show CustomerName with Total Claim Amount per Customer.*/
SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, SUM(cl.ClaimAmount)
AS TotalClaimAmount
FROM Customers c
JOIN PolicyAssignments pa
ON c.CustomerID=pa.CustomerID
JOIN Claims cl
ON pa.AssignmentID=cl.AssignmentID
GROUP BY c.FirstName, c.LastName;

```

```

/*9. Show names and total claim amount of Customers With Claim Amount > 25000
(Use HAVING Clause).*/
SELECT CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName, SUM(cl.ClaimAmount)
AS TotalClaimAmount
FROM Customers c
JOIN PolicyAssignments pa
ON c.CustomerID=pa.CustomerID
JOIN Claims cl
ON pa.AssignmentID=cl.AssignmentID
GROUP BY c.FirstName, c.LastName
HAVING SUM(cl.ClaimAmount)>25000;

```

```

/*10. Display list with Agent Wise Policy Count.*/
SELECT a.AgentID, a.AgentName, COUNT(pa.PolicyID) AS PolicyCount
FROM Agents a
LEFT JOIN PolicyAssignments pa
ON a.AgentID=pa.AgentID
GROUP BY a.AgentID, a.AgentName;

```

30-12-2025 (Day-3)

Subqueries

Subqueries using EXISTS

--1. Find customers who have at least one policy.

```
SELECT c.*
FROM Customer c
WHERE EXISTS(
    SELECT 1
    FROM PolicyAssignment pa
    WHERE pa.CustomerID = c.CustomerID
);
```

--2. List customers who have made at least one claim.

```
SELECT c.*
FROM Customer c
WHERE EXISTS(
    SELECT 1
    FROM PolicyAssignment pa
    JOIN Claim cl
    ON cl.AssignmentID = pa.AssignmentID
    WHERE pa.CustomerID = c.CustomerID
);
```

--3. Find policies for which at least one claim exists.

```
SELECT p.PolicyID, p.PolicyName
FROM Policies p
WHERE EXISTS(
    SELECT 1
    FROM PolicyAssignment pa
    JOIN Claim c
    ON c.AssignmentID = pa.AssignmentID
    WHERE pa.PolicyID = p.PolicyID
);
```

--4. Find customers who have a policy with premium > 5,000.

```
SELECT c.*
FROM Customer c
WHERE EXISTS(
    SELECT 1
    FROM PolicyAssignment pa
    JOIN Policies p
    ON pa.PolicyID=p.PolicyID
    WHERE pa.CustomerID=c.CustomerID
    AND p.PremiumAmount>5000
)
)
```

--5. List policies where a claim with amount > 20,000 exists.

```
SELECT p.*
FROM Policies p
WHERE EXISTS(
    SELECT 1
    FROM PolicyAssignment pa
    JOIN Claim c
    ON c.AssignmentID=pa.AssignmentID
    WHERE pa.PolicyID=p.PolicyID
    AND c.ClaimAmount>20000
);
```

Subqueries using ANY

--6. Find policies with premium greater than ANY premium of policies held by CustomerID=101.

```
SELECT p.*
FROM Policies P
WHERE p.PremiumAmount>ANY(
    SELECT p2.PremiumAmount
    FROM Policies p2
    JOIN PolicyAssignment pa
    ON pa.PolicyID = p2.PolicyID
    WHERE pa.CustomerID = 101
);
```

--7. List customers whose policy premium is greater than ANY premium in the Health policy type.

```
SELECT c.*
FROM Customer c
JOIN PolicyAssignment pa
ON c.CustomerID=pa.CustomerID
JOIN Policies p
ON pa.PolicyID=p.PolicyID
WHERE p.PremiumAmount>ANY(
    SELECT p2.PremiumAmount
    FROM Policies p2
    WHERE p2.PolicyType='Health'
);
```

--8. Display policies whose premium is less than ANY premium of policies issued after 2023.

```
SELECT p.*
FROM Policies P
WHERE p.PremiumAmount<ANY(
    SELECT p1.PremiumAmount
    FROM Policies p1
    JOIN PolicyAssignment pa
    ON pa.PolicyID = p1.PolicyID
    WHERE YEAR(pa.StartDate)>2023
);
```

--9. Find customers whose policy premium is greater than ANY premium of customers from Mumbai.

```
SELECT DISTINCT c.*
FROM Customer c
JOIN PolicyAssignment pa
ON c.CustomerID=pa.CustomerID
JOIN Policies p
ON pa.PolicyID=p.PolicyID
WHERE p.PremiumAmount>ANY(
    SELECT p1.PremiumAmount
    FROM Policies p1
    JOIN PolicyAssignment pa1
    ON pa1.PolicyID=p1.PolicyID
    JOIN Customer c1
    ON c1.CustomerID=pa1.CustomerID
    WHERE c1.City='Mumbai'
);
```

--10. Find claims whose amount is less than ANY claim rejected.

```
SELECT *
FROM Claim c
WHERE c.ClaimAmount < ANY(
    SELECT ClaimAmount
    FROM Claim
    WHERE ClaimStatus = 'Rejected'
);
```

Subqueries using ALL

--11. Find policies with premium greater than ALL premiums of Motor policies.

```
SELECT *
FROM Policies p
WHERE p.PremiumAmount > ALL(
    SELECT PremiumAmount
    FROM Policies
    WHERE PolicyType = 'Motor'
);
```

--12. List customers whose policy premium is less than ALL premiums of Life policies.

```
SELECT DISTINCT c.*
FROM Customer c
JOIN PolicyAssignment pa
ON c.CustomerID = pa.CustomerID
JOIN Policies p
ON pa.PolicyID = p.PolicyID
WHERE p.PremiumAmount < ALL(
    SELECT PremiumAmount
    FROM Policies
    WHERE PolicyType = 'Life'
);
```

--13. Find claims where claim amount is greater than ALL claims made in 2023.

```
SELECT c.*
FROM Claim c
WHERE c.ClaimAmount > ALL (
    SELECT ClaimAmount
    FROM Claim
    WHERE YEAR(ClaimDate) = 2023
);
```

--14. Find agents whose policy premiums are greater than ALL premiums handled by AgentID = 3.

```
SELECT DISTINCT a.AgentID, a.AgentName
FROM Agents a
JOIN PolicyAssignment pa
ON a.AgentID = pa.AgentID
JOIN Policies p
ON pa.PolicyID = p.PolicyID
WHERE p.PremiumAmount > ALL(
    SELECT p2.PremiumAmount
    FROM Policies p2
    JOIN PolicyAssignment pa2
    ON pa2.PolicyID = p2.PolicyID
    WHERE pa2.AgentID = '3'
);
```

```
--15. Display policies whose premium is less than ALL premiums of policies
having claims.
SELECT p.*
FROM Policies p
WHERE p.PremiumAmount < ALL(
    SELECT p1.PremiumAmount
    FROM Policies p1
    JOIN Claim c
    ON c.PolicyID=p1.PolicyID
);
```

SET Operators in SQL Server

i) UNION

```
--UNION
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID=pa.CustomerID

UNION

SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID=pa.CustomerID
JOIN Claim cl ON cl.AssignmentID=pa.AssignmentID;
```

ii) UNION ALL

```
--UNION ALL
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID=pa.CustomerID

UNION ALL

SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID=pa.CustomerID
JOIN Claim cl ON cl.AssignmentID=pa.AssignmentID;
```

iii) INTERSECT

```
--INTERSECT
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID=pa.CustomerID

INTERSECT

SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID=pa.CustomerID
JOIN Claim cl ON cl.AssignmentID=pa.AssignmentID;
```

iv) EXCEPT

```
--EXCEPT
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID=pa.CustomerID

EXCEPT

SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID=pa.CustomerID
JOIN Claim cl ON cl.AssignmentID=pa.AssignmentID;
```

Some Queries using SET Operator

```
--1. Customers who bought Life OR Health policies
SELECT DISTINCT c.CustomerID, c.FirstName, p.PolicyType
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID = pa.CustomerID
JOIN Policies p ON p.PolicyID = pa.PolicyID
WHERE p.PolicyType = 'Life'
```

UNION

```
SELECT DISTINCT c.CustomerID, c.FirstName, p.PolicyType
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID = pa.CustomerID
JOIN Policies p ON p.PolicyID = pa.PolicyID
WHERE p.PolicyType = 'Health';
```

```
--2. Customers who bought Life but NOT Health
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID = pa.CustomerID
JOIN Policies p ON p.PolicyID = pa.PolicyID
WHERE p.PolicyType = 'Life'
```

EXCEPT

```
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID = pa.CustomerID
JOIN Policies p ON p.PolicyID = pa.PolicyID
WHERE p.PolicyType = 'Health';
```

```
--3. All customers with policies (allow duplicates)
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID = pa.CustomerID
```

UNION ALL

```
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID = pa.CustomerID;
```

```

--4. Common policies in Claim and Assignment
SELECT p.PolicyID, p.PolicyName, p.PolicyType
FROM Policies p
JOIN PolicyAssignment pa ON p.PolicyID = pa.PolicyID

INTERSECT

SELECT p.PolicyID, p.PolicyName, p.PolicyType
FROM Policies p
JOIN PolicyAssignment pa ON p.PolicyID = pa.PolicyID
JOIN Claim c ON c.AssignmentID = pa.AssignmentID;

--5. Customers with policies but no claims.
SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID = pa.CustomerID

EXCEPT

SELECT c.CustomerID, c.FirstName, c.City
FROM Customer c
JOIN PolicyAssignment pa ON c.CustomerID = pa.CustomerID
JOIN Claim cl ON cl.AssignmentID = pa.AssignmentID;

```

3.1 CASE...ELSE

```

--1. Categorize Premium Level of Policies
SELECT PolicyID, PolicyName, PremiumAmount,
       CASE
           WHEN PremiumAmount >= 20000 THEN 'Diamond Premium'
           WHEN PremiumAmount BETWEEN 10000 AND 19999 THEN 'Gold Premium'
           ELSE 'Sliver Premium'
       END AS Premium_Category
FROM Policies;

--2. Show Claim Status Meaningfully
SELECT ClaimID, ClaimAmount, ClaimStatus,
       CASE
           WHEN ClaimStatus = 'Approved' THEN 'Claim Successfully Settled'
           WHEN ClaimStatus = 'Rejected' THEN 'Claim Denied'
           ELSE 'Pending Review'
       END AS Status_Remark
FROM Claim;

```

3.2 Merge in SQL Server

```

--Staging Table Creation
CREATE TABLE Agents_Staging(
    AgentID VARCHAR(50) PRIMARY KEY,
    AgentName VARCHAR(50),
    Phone INT,
    City VARCHAR(50)
)

```

```

--Insert data into Staging Table
INSERT INTO Agents_Staging(AgentID, AgentName, Phone, City)
VALUES
('A101', 'Rohit Verma', 987651234, 'Hyderabad'),
('A102', 'Sneha Kapoor', 998866554, 'Delhi'),
('A104', 'Arjun Sharma', 900087569, 'Kolkata'),
('A105', 'Ayush Reddy', 987654321, 'Hyderabad');

--Displaying Initial Tables
SELECT * FROM Agents;
SELECT * FROM Agents_Staging;

--Performing MERGE
MERGE Agents AS a
USING Agents_Staging AS s
ON a.AgentID=s.AgentID
WHEN MATCHED THEN
    UPDATE SET
        a.AgentName=s.AgentName,
        a.Phone=s.Phone,
        a.City=s.City
WHEN NOT MATCHED BY TARGET THEN
    INSERT (AgentID, AgentName, Phone, City)
    VALUES (s.AgentID, s.AgentName, s.Phone, s.City)
WHEN NOT MATCHED BY SOURCE THEN
    DELETE;

--Displaying Tables after MERGE
SELECT * FROM Agents;
SELECT * FROM Agents_Staging;

```

3.3 ROLLUP

```

--1. ROLLUP on ClaimStatus and ClaimDate
SELECT ClaimStatus, YEAR(ClaimDate) AS ClaimYear, SUM(ClaimAmount) AS
TotalClaimAmount
FROM Claim
GROUP BY ROLLUP(ClaimStatus, YEAR(ClaimDate));

--2. ROLLUP on ClaimStatus and ClaimDate GROUPING() (Readable Report)
SELECT
    CASE
        WHEN GROUPING(ClaimStatus) = 1 THEN 'ALL STATUS'
        ELSE ClaimStatus
    END AS ClaimStatus,

    CASE
        WHEN GROUPING(YEAR(ClaimDate)) = 1 THEN 'TOTAL FOR STATUS'
        ELSE CAST(YEAR(ClaimDate) AS VARCHAR(10))
    END AS ClaimYear,

    SUM(ClaimAmount) AS TotalClaimAmount
FROM Claim
GROUP BY ROLLUP(ClaimStatus, YEAR(ClaimDate));

```


3.4 CUBE

```
--CUBE on ClaimStatus and ClaimDate
SELECT ClaimStatus, YEAR(ClaimDate) AS ClaimYear, SUM(ClaimAmount) AS
TotalClaimAmount
FROM Claim
GROUP BY CUBE(ClaimStatus, YEAR(ClaimDate));

--CUBE on ClaimStatus and ClaimDate GROUPING() (Readable Report)
SELECT
    CASE
        WHEN GROUPING(ClaimStatus) = 1 THEN 'ALL STATUS'
        ELSE ClaimStatus
    END AS ClaimStatus,

    CASE
        WHEN GROUPING(YEAR(ClaimDate)) = 1 THEN 'TOTAL FOR STATUS'
        ELSE CAST(YEAR(ClaimDate) AS VARCHAR(10))
    END AS ClaimYear,

    SUM(ClaimAmount) AS TotalClaimAmount
FROM Claim
GROUP BY CUBE(ClaimStatus, YEAR(ClaimDate))
ORDER BY ClaimStatus, YEAR(ClaimDate);
```

3.5 Grouping Sets

```
--GROUPING SETS on ClaimStatus and ClaimDate
SELECT ClaimStatus, YEAR(ClaimDate) AS ClaimYear, SUM(ClaimAmount) AS
TotalClaimAmount
FROM Claim
GROUP BY GROUPING SETS(
    (ClaimStatus, YEAR(ClaimDate)),
    (ClaimStatus),
    (YEAR(ClaimDate)),
    ()
);

--GROUPING SETS on ClaimStatus and ClaimDate GROUPING() (Readable Report)
SELECT
    CASE
        WHEN GROUPING(ClaimStatus) = 1 THEN 'ALL STATUS'
        ELSE ClaimStatus
    END AS ClaimStatus,

    CASE
        WHEN GROUPING(YEAR(ClaimDate)) = 1
            AND GROUPING(ClaimStatus) = 0 THEN 'TOTAL FOR STATUS'
        WHEN GROUPING(YEAR(ClaimDate)) = 1
            AND GROUPING(ClaimStatus) = 1 THEN 'GRAND TOTAL'
        ELSE CAST(YEAR(ClaimDate) AS VARCHAR(10))
    END AS ClaimYear,
    SUM(ClaimAmount) AS TotalClaimAmount
FROM Claim
GROUP BY GROUPING SETS(
    (ClaimStatus, YEAR(ClaimDate)),
    (ClaimStatus),
    (YEAR(ClaimDate)),
    ()
)
ORDER BY ClaimStatus, ClaimYear;
```